

THERE’S MORE THAN ONE WAY: MODE COLLAPSE IN RLVR & MODEBENCH

Anonymous authors

Paper under double-blind review

ABSTRACT

A model can answer more reliably while producing a narrower range of successful outputs. With binary rewards, reinforcement learning with verifiable rewards (RLVR) gives a repeated success and a different success the same feedback. We introduce MODEBENCH, five executable domains that identify distinct verified outputs for the same prompt, and ReplayMaxRL, which combines learning from fresh samples with memory of verified successes. The same execution keys serve both measurement and replay: each retained key receives equal weight, even when it becomes rare in fresh rollouts. Across three model scales, adding replay to Dr.GRPO improves held-out correctness and sampled verified support. Replay also complements MaxRL across all three scales, with five paired seeds in every domain. Harder-level comparisons cover four complete domains, with PantryPlan remaining partial. Hosted-model evaluation also finds accurate answers concentrated on few verified modes. An idealized analysis explains why binary rewards can amplify common modes and how replay can counter that pressure. Together, these results distinguish reliable answering from the availability of verified alternatives.

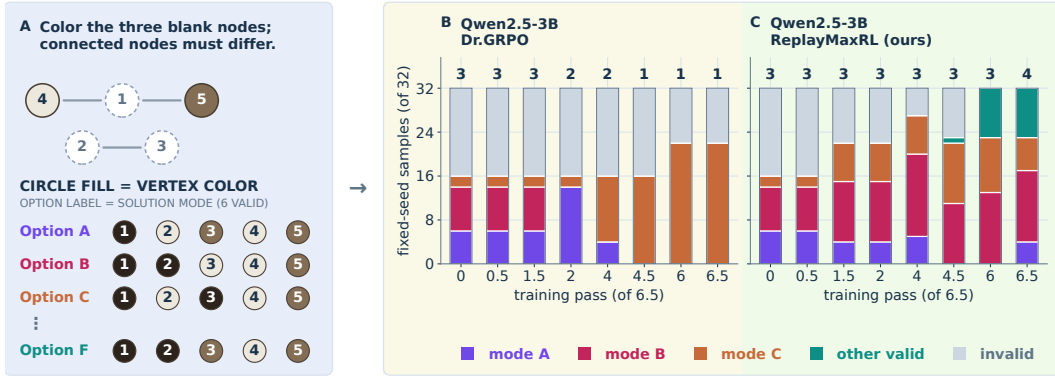


Figure 1: **Correctness can rise while sampled alternatives narrow.** One graph admits six valid colorings. Bars pool four eight-sample draws from matched Qwen2.5-3B runs. At pass 6.5, Dr.GRPO produces 22/32 correct outputs from one mode; ReplayMaxRL produces 32/32 across four. Circle fills mark vertex colors; dashed rings mark vertices left uncolored in the prompt. This illustrates the distinction; Figure 6 separates the effects of the fresh objective and replay.

1 INTRODUCTION

A language model can satisfy the same request in many ways: it can offer different explanations, generate different plans, or write different programs that meet the same requirements. These alternatives give users choices and may remain useful as preferences or constraints change. As a model becomes more successful, what happens to the diversity of successful outputs it can produce?

We study this question in reinforcement learning with verifiable rewards (RLVR), which improves a model using executable correctness feedback (Guo et al., 2025). Binary rewards treat a familiar correct solution and a different correct solution alike. Figure 1 illustrates why correctness and

verified alternatives must be measured separately. Our saved-output analysis directly finds increasing correct-key concentration in Graph and PantryPlan under Dr.GRPO and GRPO at all three scales (Appendix H.1). The raw-breadth precheck appears in Appendix H.2. The illustration compares different fresh objectives; the matched factorial separates their effects from replay.

MODEBENCH supplies the missing measurement: each validator returns a canonical execution key alongside correctness. Different strings may share a key; different accepted computations may reach the same answer. These identities also organize memory. ReplayMaxRL combines fresh MaxRL updates with replay of one verified exemplar per admitted key, assigning equal weight regardless of discovery frequency. Common successes therefore do not dominate the bank’s weights simply because they are sampled repeatedly.

The benchmark, conditional theory, and controlled comparisons study the same verified alternatives. Replay improves held-out success and sampled breadth in demonstrated settings and reduces conditional concentration in the Graph/Pantry comparisons under Dr.GRPO. Hosted observations establish the measurement’s broader relevance without identifying a training cause. Whether these alternatives improve later search or adaptation remains untested.

2 MODEBENCH: MEASURING SUCCESSFUL ALTERNATIVES

For a prompt x with specification s_x , a validator maps an output y to failure $\perp$ or a canonical key $V(y, s_x) \in \mathcal{C}_x$. Two accepted outputs count as the same mode exactly when their keys agree. Training sees only keys generated by the current policy; neither the full valid-key set $\mathcal{C}_x$ nor its size is supplied to the learner or replay bank.

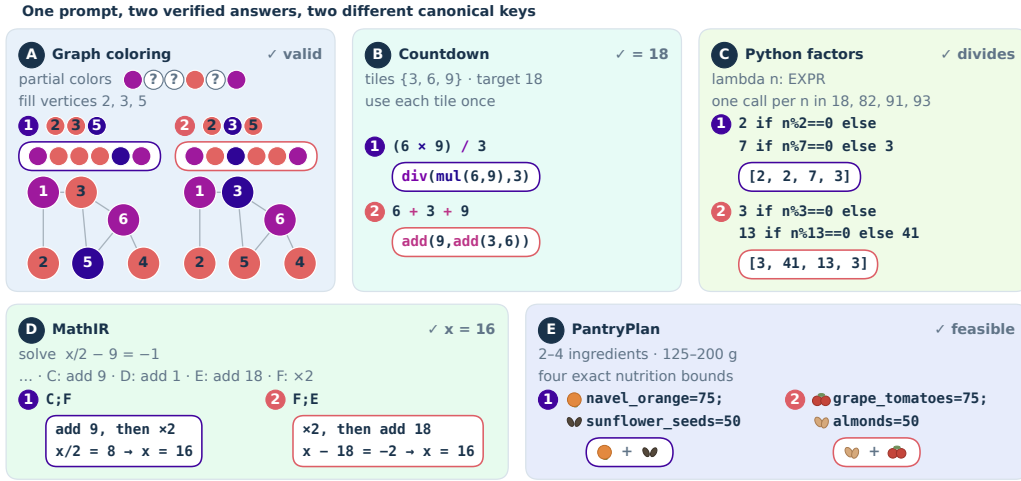


Figure 2: **One prompt can have different verified outputs.** Keys identify a color vector in Graph, a normalized computation tree in Countdown, an executed return vector in Python, an algebraic state trajectory in MathIR, or an ingredient support in PantryPlan. Cosmetic aliases merge; different accepted computations or configurations remain distinct under each domain’s execution rule.

Figure 2 shows the five domains. Countdown and MathIR capture different routes to an answer, Python captures different tested behaviors, and Graph and PantryPlan capture different valid configurations. This distinction matters when interpreting breadth: an additional valid configuration and an additional computation route are both verified modes, but they are not the same kind of alternative. The keys measure executed outputs, not latent reasoning strategies or the usefulness of a solution in a later task. Appendix D gives the exact key and alias rules, with verbatim prompts in Appendix E.

2.1 SUCCESS AND SAMPLED BREADTH

Let $P = \Pr[V(Y, s_x) \neq \perp]$ and, when $P > 0$, let $q_c = \Pr[V(Y, s_x) = c \mid V(Y, s_x) \neq \perp]$. For K independent samples, D_K counts their distinct correct keys. The promptwise identities are

$$\underbrace{\Pr[D_K > 0]}_{\text{pass@K}} = 1 - (1 - P)^K, \quad \underbrace{\mathbb{E}[D_K]}_{\text{distinct@K}} = \sum_c [1 - (1 - Pq_c)^K]. \quad (1)$$

The first endpoint measures success probability; the second measures an expected number of verified keys. We use $K = 8$ for the training endpoints. Their difference, *extra modes*, counts verified keys beyond the first success. Both raw breadth and extra modes can increase because correctness increases; neither by itself establishes a more balanced q .

Conditional collision measures that allocation directly:

$$C(q) = \sum_c q_c^2, \quad B_2 = 2P - P^2 C(q), \quad (2)$$

where $B_2 = \mathbb{E}[D_2]$. Thus C is the probability that two independent correct outputs share a key. Greater C means greater concentration. We call an observed increase in this conditional concentration *mode collapse*; finite samples do not establish literal extinction of unobserved modes. At fixed P , majorization toward a more concentrated q lowers expected breadth, but a change in C alone does not imply majorization or a change in total support (Appendix B).

2.2 HARDER TASKS WITH THE SAME NOTION OF A MODE

Levels retain each domain’s validator and canonical keys while changing problem structure. Level 1 is the original construction. Level 2 increases structural demands, such as hidden graph vertices and required Countdown operands, while preserving designated valid-mode-count histograms. Before training, frozen Qwen2.5-0.5B development pass@8 must lie in $[\cdot 10, \cdot 90]$ and below its Level-1 value in every domain (Figure 3).



Figure 3: Level construction is checked before training. Historical frozen Qwen2.5-0.5B development pass@8 for Levels 1 and 2. Level 2 lowers success in every domain while staying within the admission band $[\cdot 10, \cdot 90]$ (shaded). These construction checks are separate from held-out training results and the planned five-level evaluation across Qwen model sizes.

The scale-calibrated extension uses Qwen2.5-3B, 7B, and 14B for Levels 3, 4, and 5, respectively. Each targets the same fixed Qwen2.5-0.5B Level-1 success rates; distinct@8 is measured but never fitted. Fresh held-out confirmation admits a level only if all five domains meet the registered pass@1 and pass@8 tolerances. Level 3 with its original prompts is admitted; a neutral Python revision and Levels 4–5 remain under calibration. These construction checks precede the replay comparison in Experiment 3. Native training evaluations use separate populations and parts of the protocol, so cross-level differences do not isolate difficulty (Appendix D.3).

3 REPLAYMAXRL: LEARN FROM NEW SAMPLES AND REMEMBER SUCCESSES

Figure 4 shows the two uses of verification: correctness rewards train on fresh samples, and canonical keys organize memory. Both paths use policy-generated responses; neither requires gold solutions or a precomputed catalogue of valid modes.

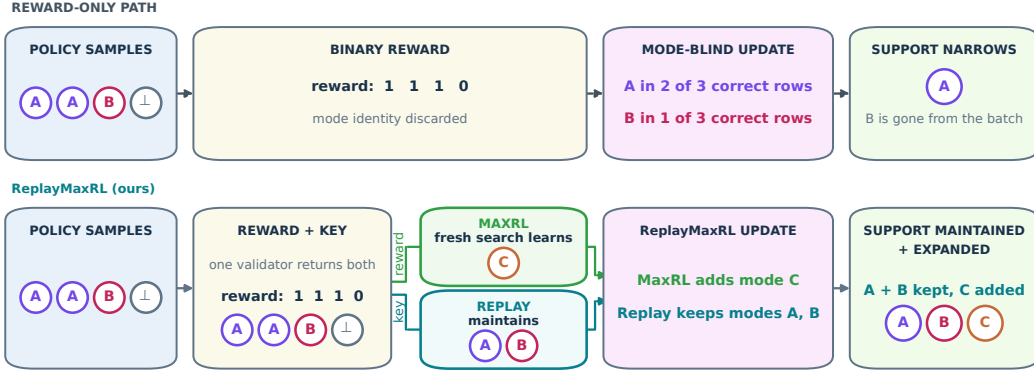


Figure 4: **Fresh learning and memory use the same verified samples.** MaxRL learns from their binary rewards. Replay stores exemplars by canonical key and revisits retained keys with equal weight. In the schematic, fresh samples add C while replay revisits A and B. A stored exemplar can continue to receive a training signal after it becomes rare in fresh samples.

3.1 LEARNING FROM FRESH ROLLOUTS

MaxRL combines success objectives across sampling budgets (Tajwar et al., 2026). With binary rewards, its centered group estimator weights a correct response more strongly when successes in the group are scarce. This provides a useful test of replay: memory should be evaluated alongside an objective designed for learning from sampled successes, as well as alongside Dr.GRPO (Liu et al., 2025).

Both fresh objectives still depend on what the policy currently samples. An absent mode contributes no direct score term, and an all-correct or all-incorrect group has zero centered task advantage. MaxRL changes the fresh learning signal without creating a persistent record of which correct behaviors have occurred. Appendix F.1 specifies the exact advantages and their finite-group interpretation.

3.2 REPLAYING DISTINCT VERIFIED SUCCESSES

Each training prompt owns an initially empty, bounded bank $\mathcal{B}_x$. When a new key is admitted, the bank retains one verified exemplar e_c . Repeated discoveries of that key do not add further entries. A deterministic schedule revisits nonempty prompt banks throughout training. For an exemplar of token length $|e_c|$, the replay loss is

$$L_{\text{replay}}(x) = -\frac{1}{|\mathcal{B}_x|} \sum_{c \in \mathcal{B}_x} \frac{\log \pi_{\theta}(e_c | x)}{|e_c|}. \quad (3)$$

Each retained key receives the same share of replay, independent of how often it appeared in fresh rollouts. A mode discovered once therefore continues to receive a signal alongside a mode discovered repeatedly. Token normalization sets the loss scale for each exemplar; it does not make exemplar likelihood equal to the total probability of generating its canonical key.

We add this loss to the fresh MaxRL update. Replacing MaxRL with Dr.GRPO yields ReplayDr.GRPO. Replay exemplars stay outside the fresh reward group, so the fresh objective and replay can be varied separately. The controls execute the same bank and scoring code with zero replay gradient; their learned banks and workloads can differ as policies diverge. Banks contain only responses to training prompts. Held-out evaluation therefore tests what training changes in the policy, rather than retrieving answers from the bank. Appendix F gives admission, scheduling, masking, and loss scaling.

3.3 WHY MEMORY ADDRESSES A DIFFERENT PRESSURE

Consider one isolated prompt, finitely many categories with at least two correct and one incorrect outcome, finite initial logits, and group size $G \geq 2$. With independent canonical-outcome logits,

common length normalization, and Euclidean infinitesimal on-policy updates without a competing regularizer, the GRPO, Dr.GRPO, and binary MaxRL mean flows satisfy, after a positive time change,

$$\frac{dq_c}{d\tau} = q_c(q_c - C), \quad \frac{dC}{d\tau} = 2 \text{Var}_{c \sim q}(q_c) \geq 0. \quad (4)$$

A unique initial winner attracts the conditional mass (Theorem C.3). Aggregating strings into keys does not preserve neural gradient geometry; this does not certify AdamW/PPO or variable-length GRPO trajectories in the implemented neural models.

For the categorical replay surrogate, bank mass P_B and its conditional allocation q_B give the identity

$$-\frac{1}{|B|} \sum_{c \in B} \log p_c = -\log P_B + \log |B| + \text{KL}(U_B \| q_B). \quad (5)$$

The objective rewards banked mass and penalizes imbalance. The actual exemplar loss is not an exact key-probability objective. In the stated categorical flow, a fixed bank and constant positive replay dose give banked modes a probability floor; unbanked modes may vanish (Theorem C.6). Neither this floor nor a bounded exemplar surrogate guarantees visibility in eight draws or held-out transfer. Appendix A maps each claim to its basis and scope.

4 EXPERIMENTAL DESIGN

We compare Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B on all five Level-1 domains. Each run trains for eight passes with five registered seeds per comparison. Evaluation averages four eight-sample draws on 128 held-out prompts, with matched requests across methods. Complete paired blocks receive nominal 95% intervals; incomplete intersections retain their exact seed counts and are descriptive. Missing or ambiguous endpoints are not imputed. The shared recipe and source-integrity rules are in Appendices D.4 and M.1.

The comparisons answer three questions. First, does adding canonical replay to Dr.GRPO improve correctness and verified support across scales? Supporting controls test reward normalization, on-policy redistribution, ordinary success memory, and semantic exploration. UCPO and sparse RLEP-Dr are compared at 0.5B and 1B only; fixed Semantic-MaxEnt is defined in Appendix F.5. Second, does replay still help when the fresh objective is MaxRL? The central factorial crosses Dr.GRPO/MaxRL with replay off/on at all three scales, yielding Dr.GRPO, ReplayDr.GRPO, MaxRL, and ReplayMaxRL. Third, does the same pattern hold on the harder Level 2? This factorial uses Qwen2.5-0.5B, with four complete domain comparisons and partial PantryPlan results. Appendix Table 6 gives the exact coverage.

5 RESULTS

Mode collapse is measurable during training. All twelve Graph/Pantry Dr.GRPO/GRPO initial-to-final contrasts increase correct-key collision, with nominal intervals above zero. Both stream orientations agree in eleven; Falcon Graph GRPO is the exception. ReplayDr.GRPO lowers collision in all six matched Graph/Pantry contrasts, with negative intervals and both orientations agreeing. MaxRL replay has negative primary contrasts there, but Falcon and 3B Graph have an orientation exception. These show mitigation, not restoration of the initial distribution (Appendix Figure 19).

For $R \geq 2$ correct outputs with counts n_c , we estimate $\hat{C} = \sum_c n_c(n_c - 1)/[R(R - 1)]$ and weight jointly eligible prompts equally within seed. Under the audited runtime mapping, four eight-output groups yield eleven nominal streams; we take each stream’s earliest output and check disjoint 5/6-stream orientations. Distinct IDs do not prove iid sampling, so these selected-population means remain descriptive. Python has no jointly eligible Dr.GRPO/GRPO initial prompts; some MathIR conditions already have collision one. Replay’s eligible-prompt correctness can decrease while full-test `pass@8` rises. Full-test endpoints retain their intact eight-output groups. Appendix H.1 reports coverage, all 135 contrasts, and the retrospective source-census extension.

5.1 EXPERIMENT 1: REPLAY IMPROVES SUCCESS AND SAMPLED SUPPORT

Replay improves success and sampled support across scales. Figure 5A shows positive block-mean changes in both primary endpoints for every model and domain. Raw `distinct@8`

increases in all 74 admissible seed pairs. This consistency matters: the result spans valid configurations, computation routes, and tested program behaviors, rather than depending on a single task or favorable seed. The excluded Falcon Countdown run and its four-seed comparison remain explicit in the figure and Appendix M.1.

A ReplayDr.GRPO (ours) – Dr.GRPO across three model scales

$\Delta \text{ pass@8}$							$\Delta \text{ distinct@8}$						
Qwen 0.5B	+0.65	+0.19	+0.36	+0.28	+0.21	+0.34	Qwen 0.5B	+2.11	+1.15	+0.40	+0.32	+1.02	+1.00
Falcon 1B	+0.15	+0.31 [†]	+0.98	+0.64	+0.12	+0.44 [†]	Falcon 1B	+0.68	+0.42 [†]	+1.11	+0.66	+0.94	+0.77 [†]
Qwen 3B	+0.15	+0.14	+0.59	+0.41	+0.11	+0.28	Qwen 3B	+0.58	+0.33	+0.78	+0.43	+0.67	+0.56
	Graph	Countdown	Python	MathIR	Pantry	Average		Graph	Countdown	Python	MathIR	Pantry	Average

B Initial reference + alternatives – terminal Dr.GRPO at Qwen2.5-0.5B

$\Delta \text{ pass@8}$							$\Delta \text{ distinct@8}$						
Before training	+0.13	-0.38	-0.17	-0.29	+0.38	-0.07	Before training	+0.24	-0.38	-0.17	-0.29	+1.90	+0.26
ReplayDr.GRPO (ours)	+0.65	+0.19	+0.36	+0.28	+0.21	+0.34	ReplayDr.GRPO (ours)	+2.11	+1.15	+0.40	+0.32	+1.02	+1.00
MaxRL (no replay)	+0.21	+0.10	+0.00	-0.07	+0.01	+0.05	MaxRL (no replay)	+0.35	+0.13	+0.00	-0.07	+0.01	+0.09
UCPO	-0.00	+0.13	+0.00	+0.03	+0.05	+0.04	UCPO	-0.00	+0.18	+0.00	+0.03	+0.05	+0.05
RLEP-Dr	+0.10	+0.10	+0.00	+0.13	+0.00	+0.07	RLEP-Dr	+0.15	+0.13	+0.00	+0.12	+0.00	+0.08
Semantic-MaxEnt	+0.00	+0.14	+0.00	-0.12	+0.20	+0.04	Semantic-MaxEnt	+0.00	+0.18	+0.00	-0.12	+1.12	+0.24
GRPO	+0.04	+0.16	+0.00	-0.05	+0.02	+0.03	GRPO	+0.05	+0.17	+0.00	-0.05	+0.02	+0.04
	Graph	Countdown	Python	MathIR	Pantry	Average		Graph	Countdown	Python	MathIR	Pantry	Average

Figure 5: **Canonical replay improves correctness and verified breadth.** (A) ReplayDr.GRPO minus Dr.GRPO at all three scales. (B) Qwen2.5-0.5B alternatives minus Dr.GRPO. Columns measure probability points and expected verified-key counts, respectively. The green Average weights domains equally within seed. A dagger marks four paired seeds in Falcon Countdown and its Average; other cells use five. Only complete blocks receive nominal 95% intervals. Purple denotes ReplayDr.GRPO; bold entries mark column maxima in Panel B.

Countdown provides a concrete interpretation: replay more than triples the number of sampled verified computation routes at Qwen2.5-0.5B while also raising correctness. Python shows why the two metrics must be read together: its correctness improves substantially, but the gain in breadth beyond the first correct output remains uncertain. More successful samples alone do not demonstrate a more balanced distribution over correct outputs. The appendix reports the exact means and uncertainty for these examples (Appendix G.2).

The controls also distinguish the value of balancing keys from simply reusing successes. At Qwen2.5-0.5B, ReplayDr.GRPO has the largest mean correctness gain among the direct alternatives in every domain (Figure 5B). Replacing equal key weights with weights proportional to fresh discovery frequency reduces extra modes on average, while the aggregate correctness contrast remains inconclusive. The advantage is concentrated in Graph, Countdown, and PantryPlan; it is not a uniform benefit across tasks. This ablation supports key balancing as a contributor to breadth (Appendix G.6, Figure 18).

5.2 EXPERIMENT 2: REPLAY COMPLEMENTS A STRONGER FRESH OBJECTIVE

Replay adds value beyond MaxRL at all three scales. The two replay comparisons in Figure 6 separate memory from the choice of fresh objective. At 0.5B, 1B, and 3B, replay improves the domain-averaged correctness and raw mode count under both Dr.GRPO and MaxRL. Thus the replay benefit persists when fresh learning already uses MaxRL’s sampling-aware objective. The comparison identifies the effect of adding the replay update; it does not establish which individual stored modes account for the held-out gains.

The effect is not identical across domains or scales. ReplayMaxRL improves both endpoints in every 0.5B domain. At 1B, mean correctness improves in every domain, but Graph’s mean raw-support effect is negative and inconclusive. The 3B MaxRL comparison is now complete: all 50 endpoints cover five domains, both methods, and seeds 70–74. Averaging domains equally within seed, correctness rises from .547 to .701 and verified modes from .744 to 1.107. All five domains have positive mean effects on both endpoints; Graph and Python correctness inter-

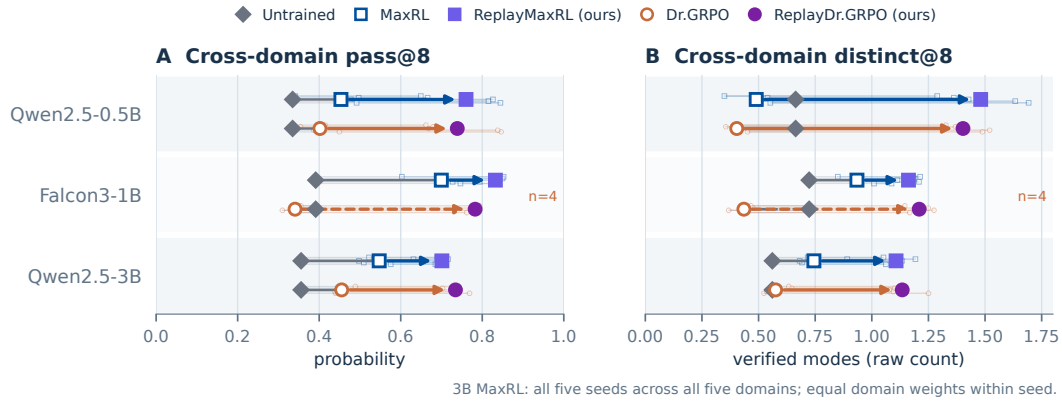


Figure 6: **Replay complements both fresh objectives at multiple scales.** Each row shows Dr.GRPO/ReplayDr.GRPO and MaxRL/ReplayMaxRL on both endpoints. Diamonds are initial models; open/filled markers are training without/with replay. Complete tracks average domains within seed; faint paths show seeds. All MaxRL tracks use five paired seeds in all five domains, including 3B. Falcon Dr.GRPO uses four common seeds; the other Dr.GRPO tracks use five. Appendix Figure 10 reports each domain’s correctness and verified-mode results at every scale.

vals still include zero. Appendix Figure 9 shows domain-specific effects; Appendix G.2 gives the aggregates; Appendix G.4 shows both metrics throughout training.

5.3 EXPERIMENT 3: REPLAY WITHIN A SECOND TASK CONSTRUCTION

Replay improves average endpoints within both levels. Figure 7 compares the four domains with complete factorials at Qwen2.5-0.5B: Graph, Countdown, Python, and MathIR (80/80 endpoints). Their aggregate moves toward greater correctness and sampled support with replay under either fresh objective, at both levels. The levels share mode definitions but use separate test populations; the within-level replay comparisons control prompts and fresh objectives. PantryPlan remains outside this aggregate because its Level-2 factorial is incomplete (10/20 endpoints).

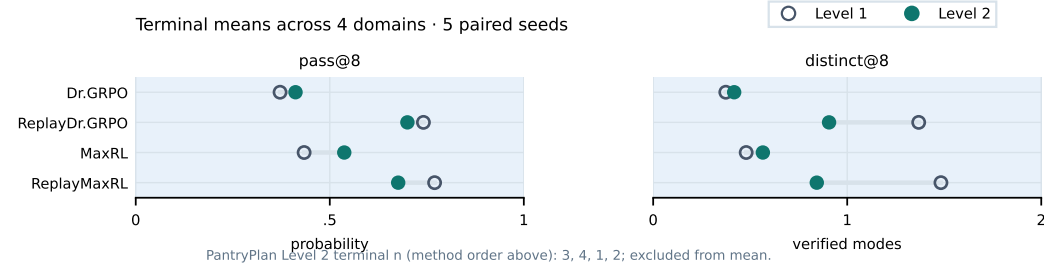


Figure 7: **Replay across difficulty levels at Qwen2.5-0.5B.** Open/filled markers denote Level 1/2. Terminal means average Graph, Countdown, Python, and MathIR equally within each of five paired seeds. Both levels and all four methods share these seeds. Construction and admission belong to the benchmark design (Section 2.2); different test populations and parts of the protocol prevent an isolated difficulty interpretation. Partial PantryPlan arm counts remain separate (Appendix G.3).

Graph supplies the clearest domain-level evidence: both replay variants improve correctness and raw support substantially on the harder problem. The other completed domains’ correctness and raw-support intervals include zero. We therefore interpret the aggregate as evidence that replay can remain useful on these harder tasks; the cross-level comparison does not isolate a causal difficulty effect or establish universal improvement. Appendix G.3 reports the Graph estimates, and Appendix G.3 gives all fresh-objective and replay-interaction contrasts. Appendix Figure 11 uses the common four-arm intersection: five seeds in each complete domain, one descriptive Pantry seed. Sparse Pantry collision contrasts increase under replay, so the concentration benefit is not universal.

Prompt guidance and sampling budget affect the observed breadth. Two Qwen2.5-0.5B follow-ups cover selected Level-2/3 Python, MathIR, and Pantry problems. Removing strategy guidance, including an executable Python example, substantially lowers Python correctness (Appendix K). Collecting 64 fresh responses per problem and wording finds persistent concentration in most trained MathIR conditions, but additional discovery in Pantry and neutral-prompt Python replay (Appendix L). Eight draws can therefore understate available alternatives; prompt-driven correctness changes alone do not establish redistribution among correct solutions.

5.4 HOSTED MODELS: CORRECTNESS DOES NOT DETERMINE BREADTH

The distinction measured by MODEBENCH also appears beyond the training comparisons. We evaluate seven hosted deployments on all five domains and three levels, without additional training. Table 1 averages `pass@8` and `distinct@8` across domains at each level. Across deployments and levels, `pass@8` ranges from 98.0% to 100%, while the mean number of verified modes ranges from 1.36 to 2.31. Appendix Figure 23 gives the domain breakdown.

Model	Level 1		Level 2		Level 3	
	<code>pass@8</code> (%)	# modes	<code>pass@8</code> (%)	# modes	<code>pass@8</code> (%)	# modes
 GPT-5.6 Sol	98.0	1.80	98.4	1.50	99.2	1.45
 Claude Opus 5	98.3	1.43	99.7	1.44	99.8	1.36
 GPT-5.4	98.1	2.11	98.4	1.72	99.5	1.68
 Grok 4.3	99.5	1.97	100.0	1.57	100.0	1.55
 Kimi K3	98.4	2.31	100.0	2.05	100.0	1.90
 Claude Opus 4.8	98.8	1.64	100.0	1.67	99.8	1.51
 DeepSeek V4 Pro	98.4	2.01	99.1	1.84	99.7	1.62

Table 1: **Hosted success and output diversity, averaged by level.** Each level averages five domains equally, using 128 prompts per domain and eight responses per prompt. `pass@8` is the fraction with any verified response; # modes is mean `distinct@8` over all prompts, including failures.

A fixed formatting normalizer applies to every deployment; Opus 5 Python uses revised task wording without a system message. All runs requested medium reasoning. Appendix J reports original-protocol results, prompt comparisons and uncertainty.

These exploratory comparisons describe the evaluated deployment and prompt configurations; they do not identify a training cause or a model-size effect. Hosted collision pools correct pairs, with population weights proportional to P^2 at a common draw budget; training collision weights eligible prompts equally. These are different aggregate estimands. Higher levels need not lower correctness for every deployment, and the display is not a common-prompt ranking.

A matched 480-prompt subset tests five temperatures with GPT-5.6 Sol (Figure 8). Non-default temperatures require reasoning `none` on this deployment. Raising T from 0 to 1.5 increases normalized empirical `pass@8` from 65.6% to 79.8% and mean `distinct@8` from .673 to 1.125. At $T = 2$, these are 79.4% and 1.146; paired 95% intervals for $T = 2$ minus $T = 1.5$ include zero on both axes.

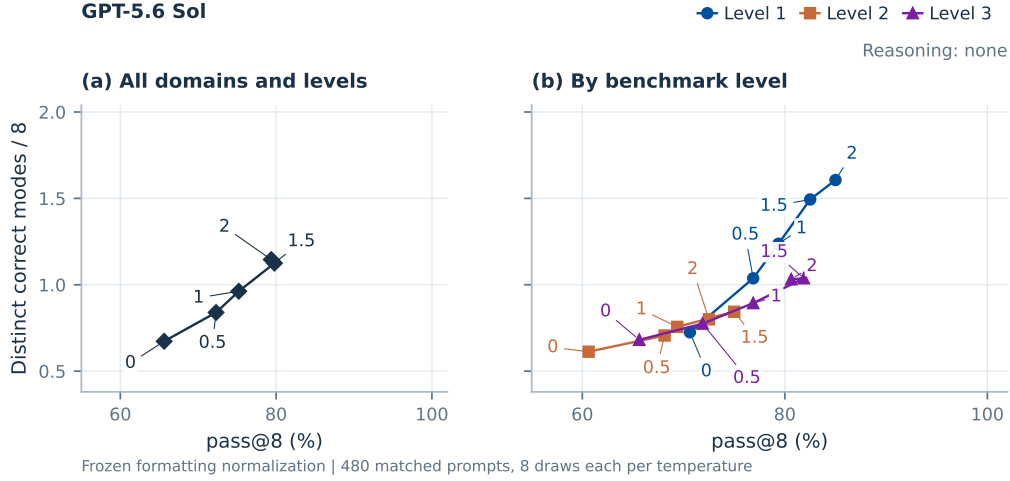


Figure 8: **Temperature traces $\text{pass}@8$ and verified breadth.** GPT-5.6 Sol, with 480 fixed prompts and eight draws each at $T \in \{0, 0.5, 1, 1.5, 2\}$. Left: all domains and levels; right: separate levels, averaging five domains equally. Empirical $\text{pass}@8$ is the fraction with at least one correct draw; $\text{distinct}@8$ averages unique correct modes, including zeroes. Connected points use reasoning none and the frozen formatting normalizer. Appendix J.3 gives pointwise uncertainty intervals for every benchmark level and all five measured temperatures.

6 RELATED WORK

RLHF and reasoning RL can reduce output diversity, entropy and high-budget coverage (Kirk et al., 2024; Dang et al., 2025; Cui et al., 2025; Yue et al., 2025). MODEBENCH measures verified output identities; semantic clustering instead infers equivalence from language (Farquhar et al., 2024). Sampling-based selection motivates preserving proposal alternatives (Brown et al., 2024; Snell et al., 2024; Cobbe et al., 2021; Lightman et al., 2024); our experiments do not measure their downstream utility.

Outcome bonuses, set prediction, distribution matching and uniform-correct objectives also encourage broader support (Song et al., 2025; Chen et al., 2025; Li et al., 2025; 2026a;b; Lochab et al., 2026). MaxRL combines objectives across sampling budgets (Tajwar et al., 2026); we ask whether memory adds value. Replay can mitigate forgetting (Lin, 1992; Rolnick et al., 2019), and RLEP reuses successful reasoning trajectories (Zhang et al., 2025). Our memory balances execution keys independently of discovery frequency, connecting replay to behavioral archives in quality-diversity methods (Mouret & Clune, 2015; Ecoffet et al., 2021).

7 CONCLUSION

MODEBENCH separates correctness from the availability of alternative verified outputs. Replay-MaxRL uses their execution identities as memory: fresh updates improve the policy on new samples, while replay keeps stored successes involved in training. Matched comparisons show that replay can improve correctness and sampled breadth, including beyond MaxRL.

Our claims concern sampled, task-defined outputs. Replay cannot retain a mode before discovering and storing it, and improved held-out breadth does not show that every stored exemplar survives. A fixed-bank audit finds improving central scores alongside a declining tail (Appendix H.4); exact mode probabilities remain unverified. The idealized theory does not certify neural training, and the experiments do not show that breadth causes the accuracy gains. Whether retaining alternatives helps models meet changing requirements or user preferences remains an open question.